

End-to-end, Inspectable Topology Optimization in Julia via the Adjoint Method

Rena Wang¹

¹*Massachusetts Institute of Technology, Dept. of Mechanical Engineering*

Project presented: May 6, 2026

Abstract— This project presents a Julia-based framework for end-to-end, inspectable topology optimization using adjoint sensitivities. The framework targets density-based compliance minimization in linear elasticity and combines finite element analysis, Helmholtz filtering, threshold projection, SIMP material interpolation, and MMA optimization. Unlike many educational or commercial topology optimization tools, the implementation exposes the full computational pipeline, allowing custom design domains, boundary conditions, filtering parameters, projection schedules, and optimizer settings. The adjoint formulation is validated against finite-difference directional derivatives and shown to be substantially more scalable than finite-difference sensitivity computation as the number of design variables increases. Experiments on a two-dimensional cantilever beam demonstrate robust convergence across multiple initializations and produce truss-like load paths consistent with structural intuition. A three-dimensional bike-frame-inspired example further shows that the same pipeline can handle more realistic geometries and spatial load transfer. The results suggest that Julia provides a useful balance between mathematical expressiveness, implementation transparency, and computational performance for topology optimization research, and motivates future extensions to additional physics or parallel acceleration.

Keywords— Topology optimization, adjoint method, finite element method, Julia, density-based optimization

I. INTRODUCTION

TOPOLOGY Optimization is a popular computational approach in engineering, answering the question of where material should go to achieve maximum performance, given a design space and certain design constraints. Since the original concept of sensitivity-based mechanical design optimization was proposed by Bendsøe and Kikuchi (1988)[1], the field has since extended into multiple disciplines of physics (fluids, thermal, etc.) as well as many different algorithmic approaches.

Despite the field's popularity in not just academia, but also industrial simulation software like Ansys or Solidworks Simulation, very few end-to-end open source programs have been published. Of those that exist the majority are written in MATLAB and thus work with a very small set of predefined design spaces and constraints [2][3]. This means that it is impossible to run topology optimization for custom design or physics inputs, which is the case for the majority of real-world use cases. Furthermore, it is also impossible to tune optimization hyperparameters to the specific use case, limiting real usage and also optimizer performance [4].

Based on these observations, the utilization of the Julia programming language ([5]) provides several benefits. Namely, the language uses a just-in-time (JIT) compiler, meaning Julia can execute high-level code with readability like MATLAB or Python with the speed of fully compiled lower-level languages. In the context of topology optimization, where hundreds of thousands of parameters define a design, the speed benefits are crucial. At the same time, it is compatible with industry standard 3D tools like Gmsh [6] and provides preexisting topology tools via Gridap.jl [7].

Our framework pushes beyond the extremely limited Ju-

lia topology optimizer tools—namely *TopOpt.jl*[8] and *Gridap-TopOpt.jl*[9]. While these packages also leverage the computation speed and interpretability of Julia, they again only work with the design setups hard-coded into the package and use the finite-difference method for gradients, meaning computation time scales rapidly as the number of parameters increase.

In contrast, the goal of our framework will be to provide a Julia-based topology optimizer with a completely custom design area, constraints, and physics module, while providing substantial computation speedup through the adjoint method of gradient computation. As a result of this process, we expect the framework to handle problem formulations like the ones posed in the real world with high speed: including non-polygonal design areas, multiple forces/complex physics setup, and manufacturing constraints.

In Section 2, we outline the general design choices made for this framework and provide the overall algorithm. Section 3 provides the specific mathematics for the compliance-minimization problem, including the discretization of the physical system and relevant derivatives. This setup is then applied to experiments to validate the gradients, benchmark runtimes, and examine sensitivity to initialization in Section 4. We then formulate real design tasks for the optimizer in Section 5 and demonstrate the framework's viability in real-world design. Finally, conclusions are given in Section 6.

II. GENERAL METHODOLOGY

While most of the mathematics of an input system depend on the specific governing equations and objective, we begin the report with a high-level overview of the topology optimization framework and key design choices.

Algorithm 1 General Adjoint-Based Topology Optimization Loop

Require: $\Omega, \Omega_d, \Gamma, V_f, r_f, \beta, \eta$
Require: Mesh, FEM spaces, physical model, loads/sources, boundary conditions, optimizer settings, tolerance ϵ
Ensure: Optimized design p^* , physical field p_{th}^* , state solution u^*

- 1: **Notation:** $\mathcal{H}$ is the filter, $\mathcal{P}_{\beta, \eta}$ is the projection, $\mathcal{M}$ maps density to physical coefficients.
- 2: **Notation:** $R(u, m) = 0$ is the discretized PDE, J is the objective, and C denotes constraints.
- 3: Generate mesh and boundary labels
- 4: Define FEM spaces, measures, physical model, loads/sources, and boundary conditions
- 5: Initialize $p^0 = V_f$
- 6: **for** $k = 0, 1, 2, \dots$ until convergence **do**
- 7: Filter design: $p_f^k \leftarrow \mathcal{H}(p^k)$
- 8: Project design: $p_{th}^k \leftarrow \mathcal{P}_{\beta, \eta}(p_f^k)$
- 9: Map to physical coefficients: $m^k \leftarrow \mathcal{M}(p_{th}^k)$
- 10: Solve forward problem: $R(u^k, m^k) = 0$
- 11: Evaluate objective: $J^k \leftarrow J(u^k, p_{th}^k)$
- 12: Evaluate constraints: $C^k \leftarrow C(u^k, p_{th}^k)$
- 13: Compute adjoint sensitivities: $\nabla_p J^k, \nabla_p C^k$
- 14: Update with MMA: $p^{k+1} \leftarrow \text{MMA}(p^k, J^k, \nabla_p J^k, C^k, \nabla_p C^k)$
- 15: Stop if $\|p^{k+1} - p^k\| < \epsilon$
- 16: **end for**
- 17: **return** p^*, p_{th}^*, u^*

An algorithm for the general case has been written out in Algorithm 1. Here, $\mathcal{H}$ denotes the design filter, $\mathcal{P}_{\beta, \eta}$ the threshold projection, $\mathcal{M}$ the map from density to physical coefficients, $R(u, m) = 0$ the discretized governing PDE, J the objective, and C the optimization constraints.

a. Finite Element Method

Firstly, we examine the question of how to parametrize the design space. While modern topology optimization approaches this with a wide range of methods, such as topological derivatives or via the boundary of the design[3], the density-based method proposed with the original topology optimization paper [1][10] is the most common due to its easy adaptability to various different inputs. Following the principles of Finite Element Method (FEM)[11], the density method of topology optimization works by splitting a design space into tiny geometrical areas—or finite elements. The problem is then reformulated in terms of the design parameters: the densities of each finite element ($p_1, p_2, \dots, p_n$). Now, instead of solving the governing physics equations over the whole system now, FEM solves for the system response with a small approximate version over each finite element. These results are then used in an optimization algorithm to find the best set of parameters.

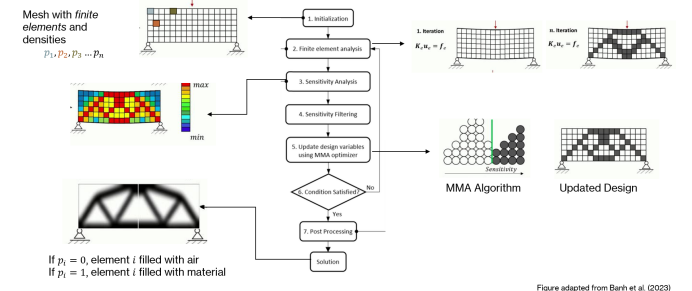
Specifically, through integration with the Gmsh Julia API [6], the design area is scriptable as a mesh or can be read directly from a CAD (3D design) file created on various external software. For easy computations in finite-element form, we use *Gridap.jl* [7], which allows for easily readable equations when working with the physics PDEs across small elements.

b. Method of Moving Asymptotes and Adjoint

As for the optimization algorithm, we choose the Method of Moving Asymptotes (MMA) first proposed by Svanberg in 1987 explicitly for structural optimization [12]. In addition to its history rooted in topology optimization, it is by far the most popular optimization method in the field. As MMA uses a local approximation rather than building a full Hessian matrix and uses the moving asymptotes to constrain the design parameters, it is able to efficiently handle many design parameters and the bounds on density, some of the key features of topology optimization. Furthermore, its implementation is well-documented in many languages, and for this project's Ju-

lia implementation we will use the algorithm straight out of *NLOpt.jl* [13].

Figure 1: A visual explanation of how finite element densities can be optimized with MMA to produce the final design. Figure adapted from [14]



To calculate the sensitivities in the optimization process, we use the adjoint method for faster computation. Specifically, we define custom pullbacks using *ChainRulesCore.jl*, which allows tools such as *Zygote.jl* [15] to propagate gradients through selected operations without differentiating through the full finite element solve as a black box.

c. Custom Constraints

Finally, we demonstrate the flexibility of our specific topology optimizer by introducing simple manufacturing constraints that ensure a machine is actually able to fabricate the optimizer's output topology. Each of these constraints include hyperparameters tunable based on the specific design inputs and machine used in fabrication.

Helmholtz Filter

The Helmholtz filter [16] smooths the raw design variables before they are used in the physics solve. However, if we use the raw densities p , the design can develop checkerboard patterns or tiny features, both of which are not realistically able to be fabricated. So, we apply a filter radius r_f , with larger r_f producing a smoother design and larger minimum feature

sizes. Then, we solve a smoothing PDE:

$$-r_f^2 \nabla^2 p_f + p_f = p \quad (1)$$

where p is the raw design variable (finite-element densities), p_f is the Helmholtz-filtered density.

Threshold Projection

After filtering however, the density field is often smooth but contains intermediate, or gray, values where $0 < p_f < 1$. In reality, we want each density to be nearly binary. Thus we apply a smooth Heaviside approximation to the filtered density field p_f to map it to the projected density p_{th} [17]:

$$p_{th}(p_f) = \frac{\tanh(\beta\eta) + \tanh(\beta(p_f - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))} \quad (2)$$

Here, $\eta \in (0, 1)$ is the threshold value: if $p_f < \eta$, the projected value is pushed toward void (0) and the opposite is also true. The parameter β controls how abrupt this transition is. As β increases, the map goes from smooth and gradual to approaching a discontinuous Heaviside function. Both η and β are tunable parameters that may vary from design to design. In fact substantial research has been done on β schemes: where we gradually increase β to transition from a general problem to a sharp, near-binary design. For this project's optimizer, we will use the doubling scheme as in Dunning and Wein [18]. Specifically, MMA will start with $\beta = 1$ and double the β value every 50 iterations up to a maximum of $\beta = 512$.

III. PROBLEM FORMULATION

For the experiments, we specialize the general topology optimization framework to a *linear elasticity compliance-minimization problem*: a widely used physics setup. The structural response is computed by solving the linear elastic finite element system $Ku = F$, where K is the density-dependent stiffness matrix, u is the displacement field, and F is the applied load.

Our objective is to minimize elastic compliance, defined by the function g , which corresponds to maximum structural stiffness under the loading. Our optimization is also constrained by a target volume fraction V_f , where the amount of material used relative to the total design area used must be less than V_f .

Putting things together, the optimization objective is:

$$\min_{p_0} g(p_0) = F^T u \quad (3)$$

$$K(p_{th}(p_f(p_0)))u = F \quad (4)$$

$$\frac{\int_{\Omega_d} p_{th} d\Omega}{|\Omega_d|} - V_f \leq 0 \quad (5)$$

$$0 \leq p_{0i} \leq 1 \quad (6)$$

Each finite element has its own density, but to relate it back to the physical system, the solver needs a material property. In elasticity problems, this is the Young's Modulus E . Here we will use the standard equation proposed by Bendsoe and Sigmund in 1999 [19]: the Solid Isotropic Material with Penalization Model (SIMP):

$$E(p_{th}) = E_{\min} + p_{th}^{q_{simp}} (E_0 - E_{\min}) \quad (7)$$

where E_0 is the solid material stiffness, $E_{\min}$ is a small residual stiffness for void-like regions, and q_{simp} is a hyperparameter representing the penalization exponent. So, in a high-density region with $p_{th} \approx 1$, $E \approx E_0$ mirroring the behavior of a solid region. Likewise, when $p_{th} \approx 0$, $E \approx E_{\min}$.

Given this non-constant E across the entire design area, we look at the stress-strain law from traditional mechanics. Along the way, it is helpful to define the Lamé parameters λ and μ from Young's modulus E and Poisson's ratio ν (a material constant independent of density).

$$\lambda(E, \nu) = \frac{E\nu}{(1 + \nu)(1 - 2\nu)} \quad \mu(E, \nu) = \frac{E}{2(1 + \nu)}$$

Following standard mechanics laws, the strain (ϵ) and stress (σ) are expressed as follows:

$$\epsilon(u) = \frac{1}{2}(\nabla u + \nabla u^T) \quad (8)$$

$$\sigma(u, E) = \lambda(E, \nu)\text{tr}(\epsilon(u))I + 2\mu(E, \nu)\epsilon(u) \quad (9)$$

a. Forward Solve

In FEM, including packages like *Gridap.jl*, it is often helpful to write the governing physics PDEs in weak form; instead of requiring the equation to hold pointwise everywhere, the weak form requires equilibrium to hold in an averaged sense against every allowable test displacement v .

To obtain the weak form equations, we work from the strong form of general linear elasticity—first without the varying density and Young's Modulus:

$$-\nabla \cdot \sigma(u) = b \quad (10)$$

where u is displacement, $\sigma(u)$ is stress, and b is a body force (a force acting on every particle of a body, rather than just its surface). For the purposes of this project, we focus on just applied forces at the surface and therefore set $b = 0$.

The boundary conditions are split into a fixed displacement boundary and a loaded traction boundary:

$$u = 0 \quad \text{on } \Gamma_D, \quad \sigma(u)n = \text{traction} \quad \text{on } \Gamma_{\text{load}}$$

with n representing the outward unit normal vector to the boundary.

To derive the weak form of Eq. 10, we multiply the strong form by an admissible test displacement v and integrate over the entire input area Ω .

$$\int_{\Omega} v \cdot (-\nabla \cdot \sigma(u)) d\Omega = \int_{\Omega} v \cdot b d\Omega \quad (11)$$

Using integration by parts:

$$\int_{\Omega} \nabla v : \sigma(u) d\Omega - \int_{\partial\Omega} v \cdot \sigma(u)n d\Gamma = \int_{\Omega} v \cdot b d\Omega \quad (12)$$

For linear elasticity, the stress tensor is symmetric, so only the symmetric part of ∇v contributes. In context, this is strain so we can substitute

$$\nabla v : \sigma(u) = \epsilon(v) : \sigma(u) \quad (13)$$

where the strain tensor ϵ follows Eq. 8. For further simplification, we note that on the fixed displacement boundary Γ_D

there is no displacement $v = 0$. Otherwise, on Γ_{load} we can substitute $\sigma(u)n = \text{traction}$.

$$\int_{\Omega} \varepsilon(v) : \sigma(u) d\Omega = \int_{\Omega} v \cdot b d\Omega + \int_{\Gamma_{\text{load}}} v \cdot \text{traction} d\Gamma \quad (14)$$

Since we previously defined $b = 0$ for this project, this simplifies to:

$$\int_{\Omega} \varepsilon(v) : \sigma(u) d\Omega = \int_{\Gamma_{\text{load}}} v \cdot \text{traction} d\Gamma \quad (15)$$

This is the generic case on linear elasticity; for topology optimization, Young's Modulus varies due to the different densities of each finite element (Eq. 7), thus affecting stress as well.

So, the generic internal stiffness term becomes:

$$\int_{\Omega_d} \varepsilon(v) : \sigma(u, E(p_{th})) d\Omega \quad (16)$$

Furthermore, for fixed ν and linear elastic problems, stress is linear in E . Applying Eq. 7:

$$\sigma(u, E(p_{th})) = \sigma(u, E_{\min}) + p_{th}^{q_{\text{simp}}} [\sigma(u, E_0) - \sigma(u, E_{\min})] \quad (17)$$

Substituting into the generic weak form from Eq. 16 gives the weak form for density-based topology optimization:

$$\begin{aligned} a(u, v; p_{th}) := & \int_{\Omega_d} \varepsilon(v) : \sigma(u, E_{\min}) d\Omega + \\ & \int_{\Omega_d} p_{th}^{q_{\text{simp}}} \varepsilon(v) : [\sigma(u, E_0) - \sigma(u, E_{\min})] d\Omega \end{aligned} \quad (18)$$

So, the final weak form becomes:

$$a(u, v; p_{th}) = \int_{\Gamma_{\text{load}}} v \cdot \text{traction} d\Gamma \quad (19)$$

Now, instead of solving for the continuous displacement, we can approximate it as a weighted sum of basis functions:

$$u_h(x) = \sum_j U_j N_j(x) \quad (20)$$

Here $N_j(x)$ is the j -th finite element basis function, which broadly determine how nodes move within the mesh, and U_j is the unknown coefficient/displacement degree of freedom associated with that basis function.

This allows us to build the stiffness matrix K used to define the system as in Eq. 4 by evaluating the weak form on basis functions:

$$K_{ij}(p_{th}) = a(N_j, N_i; p_{th})$$

In its full form, the entries of K are expressed as a function of p_{th} through:

$$\begin{aligned} K_{ij}(p_{th}) = & \int_{\Omega} \varepsilon(N_i) : \sigma(N_j, E_{\min}) d\Omega + \\ & \int_{\Omega_d} p_{th}^{q_{\text{simp}}} \varepsilon(N_i) : [\sigma(N_j, E_0) - \sigma(N_j, E_{\min})] d\Omega \end{aligned} \quad (21)$$

Similarly, the traction integral on the right-hand side of Eq.

19 can be assembled through the basis functions as well:

$$F = \int_{\Gamma_{\text{load}}} N_i \cdot \text{traction} d\Gamma \quad (22)$$

The process of assembling p_{th} through filtering and thresholding also requires conversion into the weak form: specifically the Helmholtz filter PDE. The derivation is omitted for sake of brevity; its final weak form is:

$$\int_{\Omega_d} r^2 \nabla v \cdot \nabla p_f d\Omega + \int_{\Omega_d} v p_f d\Omega = \int_{\Omega_d} v p_0 d\Omega \quad (23)$$

With these matrices assembled, the weak form becomes the forward linear system $K(p_{th})u = F$.

b. Gradients and Optimization

The optimization objective is defined by Eq. 3, where we minimize compliance (how much the structure deforms in the direction of the applied load). MMA needs the gradient of compliance wrt. the raw design variables p_0 :

$$\frac{dg}{dp_0} = \frac{dg}{du} \frac{du}{dK} \frac{dK}{dp_{th}} \frac{dp_{th}}{dp_f} \frac{dp_f}{dp_0} \quad (24)$$

Starting from the equilibrium $Ku = F$, we assume applied force F is constant and differentiate:

$$dK u + K du = 0 \quad (25)$$

With our objective $g = F^T u$, we differentiate again and simplify via substitution from Eq. 25.

$$dg = -u^T dK u \quad (26)$$

This compliance sensitivity relation means that compliance is self-adjoint, where the adjoint variable is the forward displacement solution u . So, a separate solve is not needed.

In the dependency chain shown in Eq. 24, some of the operations are pointwise scalar—with simple analytic formulas as derivatives.

$$\frac{dp_{th}}{dp_f} = \frac{\beta(1 - \tanh^2(\beta(p_f - \eta)))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))}$$

$$\frac{d(p_{th}^{q_{\text{simp}}})}{dp_f} = q_{\text{simp}} p_{th}^{q_{\text{simp}}-1} \frac{dp_{th}}{dp_f}$$

$$\frac{\partial a}{\partial p_{th}}(u, v) = q_{\text{simp}} p_{th}^{q_{\text{simp}}-1} \varepsilon(v) : [\sigma(u, E_0) - \sigma(u, E_{\min})]$$

The other derivatives involve huge Jacobians, which is why adjoints are introduced. Namely, the self-adjoint compliance in Eq. 26 accounts for the $\frac{dg}{du} \frac{du}{dK}$ terms in the chain rule. Likewise, the Helmholtz Filter PDE is handled with an adjoint solve.

In a similar manner to equations 21 and 22, p_f and p_0 are discretized using basis functions and the left and right hand sides of the weak form in Eq. 23 are assembled as matrices. This can be expressed as:

$$A_f p_f = M p_0 \quad (27)$$

So:

$$\frac{dp_f}{dp_0} = A_f^{-1} M \quad (28)$$

And the desired sensitivity equation (Eq. 24) is written as:

$$\frac{dg}{dp_0} = M^T A_f^{-T} \frac{dg}{dp_f} \quad (29)$$

To avoid intensive computation in forming the full derivative matrix, we introduce adjoint variable w

$$w = A_f^{-T} \frac{dg}{dp_f} \quad (30)$$

such that w can be found with a simple linear solve and the desired sensitivity is:

$$\frac{dg}{dp_0} = M^T w.$$

A similar process is repeated for the volume constraint in Eq. 5, reusing $\frac{dp_{vh}}{dp_0}$ from above.

The values of the objective and constraint functions, as well as their derivatives wrt. the design variable p_0 are then passed to the MMA optimizer/*NLopt.jl* to compute an update design vector:

$$p_0^{k+1} = \text{MMA} \left(p_0^k, g, \frac{dg}{dp_0}, g_V, \frac{dg_V}{dp_0} \right) \quad (31)$$

IV. EXPERIMENTS

Before applying the topology optimizer to real-world design challenges, it is important to compare the gradients and resulting design against known algorithms for accuracy. Further, to demonstrate this framework's efficiency and robustness, this section will also explore runtime benchmarking and sensitivity to initialization.

For these experiments, we consider the 2D cantilever beam shown in the figure below—a very popular toy model for topology optimization due to its simplicity. The beam will be constrained to a volume fraction of 0.4 (no more than 40% of the design area can be occupied) and subject to an applied downward force of 0.5 N on a load patch of length 0.5 m . The left edge is completely fixed, represented with a Dirichlet boundary condition. Its total design area is a rectangle of 3 $m \times 1 m$ and the material is assumed to be PLA plastic with Young's Modulus $E_0 = 2.5$ GPa.



Figure 2: 2D cantilever beam with one end fixed and the other end subject to a downward force applied to a small load patch [20]

The number of design parameters (finite elements) can be adjusted by changing the granularity of the mesh in Gmsh script or software.

a. Gradient Validation

First, before running any optimization, the gradient calculations were checked by comparing them to finite differences. This was done with a directional derivative test checking for the condition below:

$$\frac{g(p + hd) - g(p)}{h} \approx \nabla g(p)^T d \quad (32)$$

where the gradient on the right hand side is computed with the adjoint-based method described in the previous section and the left hand side is a finite difference. p is the density field and d is a perturbation direction of the same size as p ,

Throughout multiple parameter sizes and randomly generated p and d , the relative error between finite-difference and adjoint gradients **was never more than 0.01**, indicating a high level of accuracy.

Beyond individual gradients, we also compared a full MMA optimization using finite difference vs adjoint gradients to see whether the optimized design and objective/constraint values would be similar. To ensure a reasonable runtime for the algorithm, this was tested on a very small number of parameters np (35 to be exact).

Table 1: Comparison of objective and volume constraint values computed using the adjoint-based method and finite-difference verification on a coarse mesh with $np = 35$ design variables.

Quantity	Adjoint	Finite difference
Compliance objective g	13.1999	13.1639
Volume fraction	0.4000	0.3999
FD / adjoint objective		0.9973
FD / adjoint volume		0.9999

As shown in Table 1, the finite-difference and adjoint objective values agree to within approximately 0.27%, while the volume fractions agree to within approximately 0.02%. This supports that the implemented adjoint and constraint calculations are consistent with finite-difference estimates on the test case.

Figure 3: Full 500-iteration MMA optimization results on a coarse mesh with $np = 35$. A larger density corresponds to a darker color.

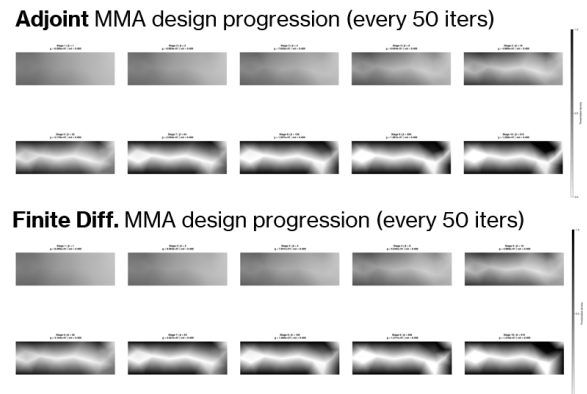


Figure 3 reinforces the consistency between adjoint and finite difference when it comes to the design itself, with the optimization producing very similar designs visually all the way through—even at the early checkpoints.

b. Runtime Comparison

Figure 4: Wall-clock time for one MMA iteration using adjoint sensitivities versus finite-difference sensitivities, logarithmic axes

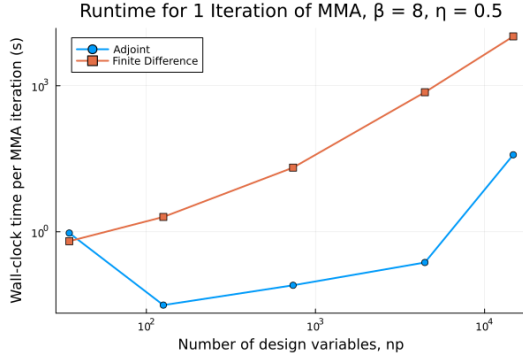
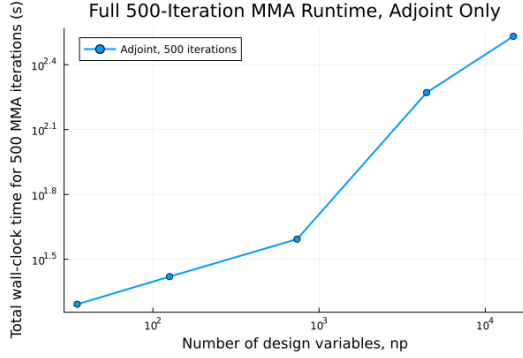


Figure 5: Wall-clock time for the full, 500 iteration run of MMA Optimization, logarithmic axes



A central motivation for using the adjoint method is that the cost of computing design sensitivities is largely independent of the number of design variables, whereas finite-difference gradients require a separate computation for each perturbation direction. To demonstrate this difference, we compared the wall-clock time required for one MMA iteration using adjoint sensitivities and finite-difference sensitivities across several mesh resolutions.

Figure 4 shows that the finite-difference runtime increases rapidly with the number of design variables np , while the adjoint method remains substantially cheaper. For the largest tested case, finite differences are orders of magnitude slower than the adjoint calculation. This confirms that finite differences are useful for small-scale verification, but are not practical as the primary sensitivity method for density-based topology optimization.

Figure 5 shows the total runtime for a full 500-iteration MMA optimization using adjoint sensitivities only. Although the total runtime still increases with mesh size due to larger finite-element systems, the adjoint formulation keeps the optimization computationally tractable at design-variable counts where finite-difference optimization would be extremely expensive. Therefore, the usage of adjoint sensitivities is demonstrated as a viable method in real-world topology optimization.

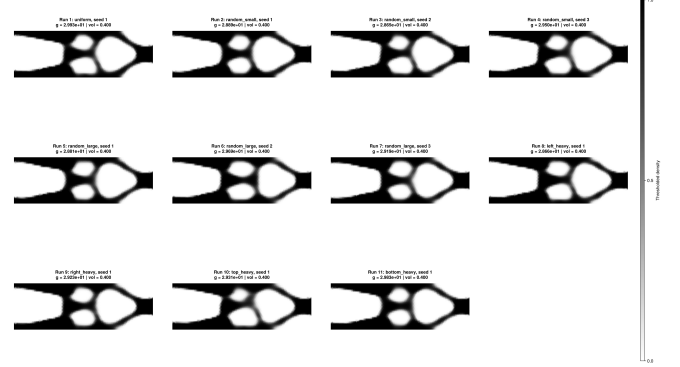
All runtime measurements were collected on a laptop equipped with an Intel Core Ultra 7 Processor 155H. Reported times therefore reflect wall-clock performance on consumer-grade hardware, as is typical of most topology optimization applications.

c. Sensitivity to Initialization

Because density-based topology optimization is nonconvex, the final design may depend on the initial density field. To evaluate whether the optimization was dominated by the choice of initial density field, we repeated the full MMA optimization using 11 distinct initializations. The tested cases included a

uniform density field, random density fields at different perturbation scales, and spatially biased initializations, all on a mesh with $np = 4436$ finite elements.

Figure 6: Final optimized designs for the various initializations used previously



First, we look at the final designs produced by each run. Upon quick visual observation of Figure 6 the final topology for all 11 initializations appear quite similar and follow a reasonable truss-like pattern from the fixed end to the load patch.

To quantitatively measure the similarity of these different initializations, we also looked at their final results across a variety of metrics. Firstly, the optimizer should converge to similar values of the objective function (compliance) as indication that the design was indeed optimal, while still satisfying the constraint. In Figure 7, compliance across the various runs are plotted, as well as final volume fraction and constraint violation (deviation from the intended volume fraction). The optimized designs produced similar final compliance values and consistently satisfied the target volume constraint. The final volume fractions are tightly clustered near $V_f = 0.4$, and the constraint values remain close to zero or negative, all indicating feasible designs.

In runtime as well, no initialization produced a significantly different runtime: with the minimum being 150 seconds and maximum being 192 seconds. Therefore, the aforementioned claim of efficiency is also robust to initialization.

Finally, the grayness metric indicates how close a density field is to being a binary black and white design—the ideal design as it is physically interpretable and manufacturable. It is defined as:

$$G(p_0) = \frac{1}{|\Omega_d|} \int_{\Omega_d} 4p_{th}(1 - p_{th}) d\Omega \quad (33)$$

So, when the material is fully solid or fully void ($p_{th} = 0$ or $p_{th} = 1$), the contribution to grayness is 0, whereas the contribution is largest at ($p_{th} = 0.5$) when there is the most ambiguity. In Figure 7, grayness is not only very close to 0 for each design, but also exhibits no significant variation across these runs.

Overall these results suggest that, for this benchmark problem and parameter set, the optimization outcome is not very sensitive to the tested initializations. However, this should not be interpreted as a general guarantee of global optimality. Since topology optimization remains nonconvex in general though, it is important to note that different design inputs like boundary conditions or design area shape may still produce initialization-dependent solutions.

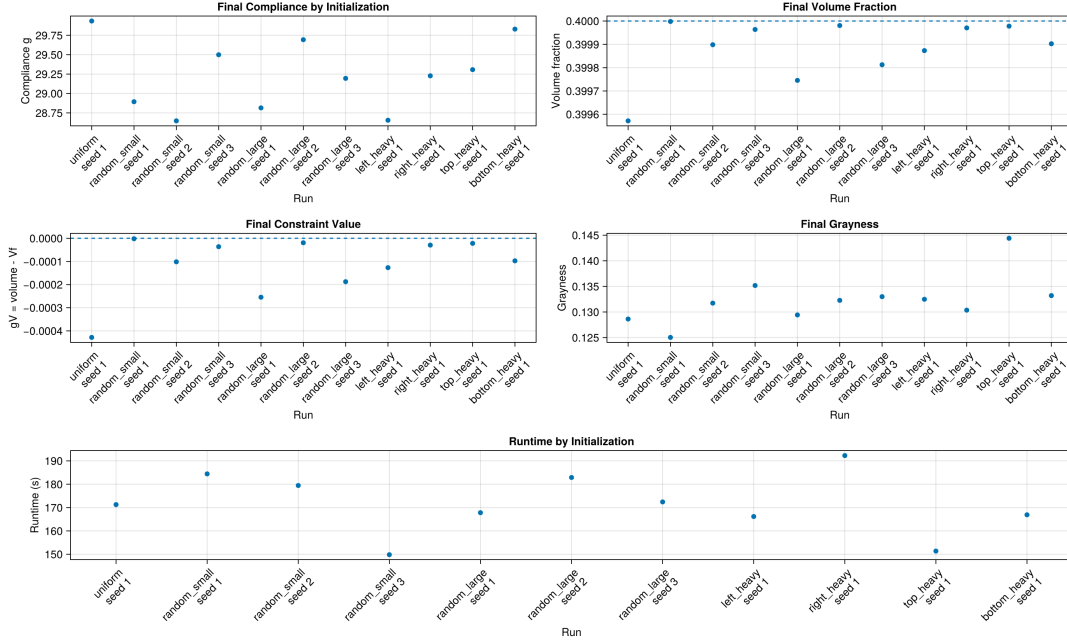


Figure 7: Converged values of various metrics, including final compliance, volume fraction, constraint value, grayness, and runtime, across 11 different initializations on the aforementioned beam setup with $np = 4436$. The dashed line on the volume fraction and constraint value plots indicates the prescribed threshold.

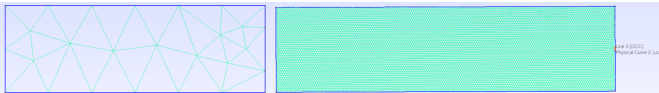
V. DESIGN TASKS AND RESULTS

Given the basic validation of the topology optimization framework, we then formulated a set of tasks across a much more complex design space and physical constraints. These tasks extend the framework beyond simple experiments to real world usages of topology optimization.

a. 2D Cantilever Beam - Fine Mesh

In the previous section, experiments were run with a low number of design parameters (coarse mesh) to ensure they could be completed in reasonable time. In reality, density-based topology optimization uses a much finer mesh to capture smooth boundaries and more intricate shapes, as is clear in Figure 8.

Figure 8: 2D beam design area for a coarse mesh of $np = 35$ (left) and fine mesh of $np = 14776$ (right). Each triangle bounded by the green lines represents 1 finite element, and the red line represents the load patch. Visualization done in Gmsh software.



For this experiment, we use the same setup shown in Figure 2, but adjust the design area and inputs to reflect a reasonable constraints for a plastic part. Specifically, the design area is a 2D rectangle with length $L = 4$ cm and height $H = 1$ cm, discretized with characteristic mesh size $l_c = 0.25$ mm—for a total of 14776 design parameters. A uniform downward traction $t = (0, -1.0)$ N/m is applied on the load patch of length 0.5 mm as shown on the right of Figure 8. The material is taken to be PLA Plastic, modeled with solid Young's modulus $E_0 = 2.5$ GPa and Poisson's ratio $\nu = 0.33$. The optimization uses a volume fraction $V_f = 0.4$, Helmholtz filter radius $r = 0.5$ mm, and $\eta = 0.5$ for thresholding. The rest of the

optimization configuration remains the same as described in Section 2.

Figure 9: Fine beam optimization results checkpointed every 50 iterations. A darker colored finite element represents higher density.

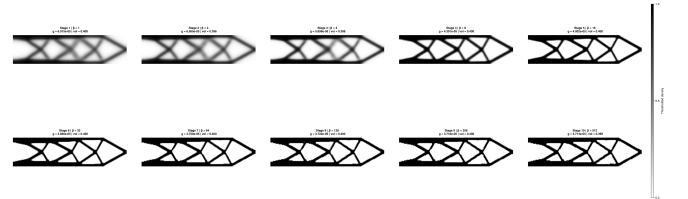
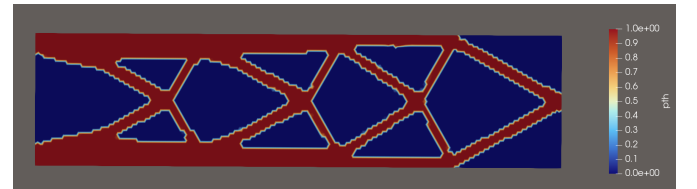


Figure 10: Final design produced by the optimizer. The high-resolution design file here is directly exported from the Julia framework and visualized in Paraview.



By qualitative check this design is quite reasonable: since the right-edge load patch applies a downward force while the left edge is fixed, the structure must transfer bending and shear loads back to the support.

The optimizer therefore retains material along diagonal truss-like paths connecting the loaded region to the fixed end, while removing material from low-strain-energy regions that contribute little to stiffness. The resulting layout resembles a lightweight braced structure: the upper and lower boundary members resist bending, while the internal diagonal members carry tension/compression and prevent excessive deflection.

Design case	Compliance g
Solid design	2.38897×10^{-5}
Uniform-density design	5.05963×10^{-4}
Void-like design	5.97244×10^1
Optimized design	3.71419×10^{-5}

Table 2: Compliance values for baseline and optimized designs.

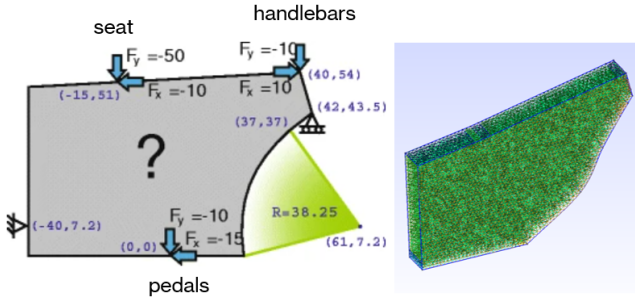
Quantitatively as well, the final design reflects a successful optimization. It finishes with a volume that is 39.95% of the total design area, under the threshold of 40%. When comparing the compliance of the optimized design against solid, uniform-density, and void-like reference cases in the table above, the optimized design achieves compliance close to the fully solid case while satisfying the imposed volume constraint, whereas the void-like and uniform-density baselines are substantially more compliant.

b. 3D Bike Frame

The 2D cantilever beam is useful for validation because it is easily interpretable. However, the value of the implementation is not only in reproducing standard compliance-minimization benchmarks, but in providing a transparent Julia framework that can be adapted to realistic structural design problems. For this reason, we extend the study to a more complex 3D design problem: the design of a bicycle frame.

Unlike the 2D beam, the bicycle geometry requires the optimizer to distribute material through a spatial volume and form load-carrying paths in three dimensions. This case tests whether the same filtering, SIMP interpolation, adjoint sensitivity, and MMA optimization pipeline can scale to a larger and more geometrically realistic topology optimization problem.

Figure 11: Bike design area and constraints adapted from [21] (left) and meshed 3D design area with $np = 56505$ finite elements (right).
Dimension units in cm and load units in kilograms-force.



Specifically, we adapt the design problem developed by Zegard and Paulino [21], meeting design area restrictions like space for the wheel, location of handlebars and pedals, and general bike size. This setup also has multiple load points, load directions, and boundary conditions (namely the two pin supports), all of which add another layer of complexity.

The design area follows the 2D profile from Figure 11 (left hand side) and has a thickness of 8cm. It is discretized with characteristic mesh size $l_c = 1.5\text{cm}$ for a total of 56505 design parameters in 3D space. The material is taken to be Aluminum, with solid Young's Modulus $E_0 = 700,000\text{kgf/cm}^2$ and $\nu = 0.3$. For optimization, we use a volume fraction of 0.3, filter radius $r = 1.2\text{cm}$, and threshold $\eta = 0.5$.

Figure 12: 3D bike frame optimization results checkpointed every 50 iterations

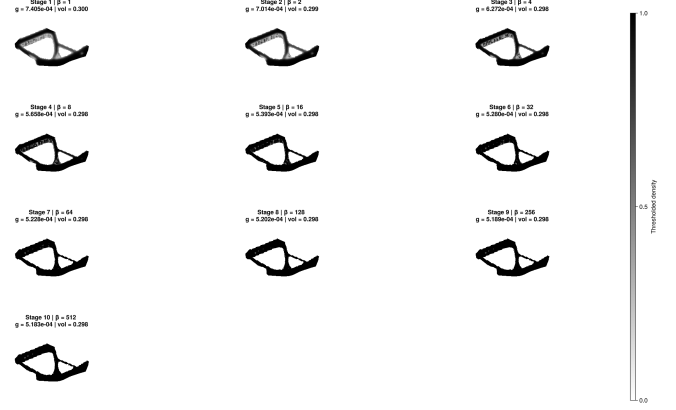
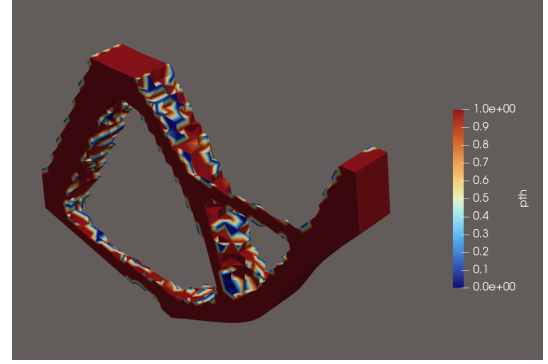


Figure 13: Optimizer's final design visualized in 3D



As expected with the optimizer, Figure 12 shows the design taking rough shape in the early iterations but with high levels of grayness. Then, as the iterations progress (and β increases per the scheme used), the design becomes smoother and much less gray. By the time it the optimization is finished, depicted in detail in Figure 13, the final design is smooth and mechanically reasonable: the optimized density field preserves material near the primary interface regions and forms continuous truss-like paths between loaded and supported boundaries to efficiently transfer load. As additional confirmation of this design, it is shown to be quite similar to bikes on the market in Figure 14.

Figure 14: Optimizer's bike design overlaid on sketch of remaining bike parts (left) versus commercially available bike (right, Rambo Rooster 2.0 bike with image from rambobikes.com)



As with the 2D cantilever problem, we perform a quantitative sanity check as well. The final design has a volume that is 29.81% the total design area volume, under the threshold of 30%. And similar to the beam, when compared to reference cases in Table 3, the optimized design has a compliance quite similar to the solid design and orders of magnitude smaller

than both the uniform and void-like designs—indicating a successful minimization of compliance given the constraints.

Table 3: Compliance values for the 3D bike frame problem.

Design case	Compliance g
Solid design	2.67678×10^{-4}
Uniform-density design	4.48626
Void-like design	2.67678×10^2
Optimized design	5.18349×10^{-4}

VI. CONCLUSION

This project implemented an adjoint-based density topology optimization framework in Julia. The optimizer combines a Helmholtz density filter, threshold projection, SIMP material interpolation, adjoint sensitivity analysis, and MMA updates to produce stiffness-efficient structures under a prescribed volume constraint. The implementation was tested on both a standard 2D cantilever benchmark and a more complex 3D bike-frame-inspired geometry, demonstrating that the same computational pipeline can be applied across increasingly realistic design domains.

The results show that the optimizer produces mechanically reasonable topologies. In the beam case, the final design forms truss-like load paths between the applied load and the fixed boundary, achieving compliance close to the fully solid reference while using substantially less material. In the 3D bike frame case, the optimizer preserves material near critical interface regions and develops internal load-carrying members through the design volume, showing that the method can generalize beyond simple planar benchmarks.

Gradient validation and sensitivity experiments further support the correctness and transparency of the adjoint-based implementation. And, in comparison to finite difference methods of computing the gradient, wall-clock benchmarking was used to show the drastic speedup in runtime by using adjoint-based gradients instead.

A central advantage of this Julia-based framework is its flexibility. Of the few Julia-based topology optimization tools that exist, none are able to accept fully custom design areas/boundary conditions/constraints. The 3D bike frame case demonstrates this framework’s ability to do just that with no changes to the implementation beyond setting up the physical inputs. Furthermore, unlike many industrial topology optimization tools, the implementation exposes algorithmic choices such as filter radius, threshold parameters, SIMP penalization, continuation schedule, initialization, and optimizer strategy. This makes the method inspectable and adaptable, allowing the effects of numerical and physical modeling choices to be studied directly.

Several limitations remain: the current formulation is restricted to linear elastic compliance minimization and is focused on one kind of optimizer, β scheme, optimization objective etc. So, additional work is needed to evaluate robustness across broader structural settings and various physics modules. If these new environments show promising results, this framework could be a good addition to the Julia package registry and provide substantially more freedom to tackle real topology optimization tasks in Julia.

Another interesting direction is GPU acceleration and HPC parallelization. A review conducted by Wang et al. [2] noted how GPU-based implementations have been more efficient

compared to their CPU-based counterparts. However, these parallelized implementations are few in number and written in low level languages like C++. These implementations are also far from mature: Zegard and Paulino’s study [21] on GPU acceleration for optimization of unstructured meshes found issues in GPU precision and convergence. On the other hand, while Aage et al. [22] was able to converge on reasonable designs and speed up the process via CPU parallelization, their implementation focuses solely on structured meshes. These structured, or grid-based, meshes cannot capture the curved boundaries and non-rectangular regions that make up most real topology optimizer inputs—such as the bike frame design. Given this project’s Julia-based framework, we could be able to balance high-level mathematical expressiveness and low-level performance control through packages like *CUDA.jl* [23]. Especially as the field of topology optimization is still evolving, Julia offers the flexibility to quickly add/change elements of the optimizer while still being able to use the GPU.

All code for this project can be found at https://github.com/renaww/top_opt_18337.

TOOL-USE REFLECTION

Before 18.337, most of my AI usage was through chatbots like ChatGPT or Gemini’s chat interface. For this project as well, I didn’t find a reason to use anything more agentic than the browser chats. Specifically, most of my AI use was ChatGPT 5.5 in the browser, since I had a subscription. I briefly tried Gemini (Gemini 3 thinking mode) as well and had GitHub Copilot’s inline suggestions turned on while coding in VSCode.

I didn’t really find the latter two AI assistants to be super helpful. In the case of Gemini, I did find the answers it gave to be more succinct/straight to the point, perhaps because I didn’t have a paid plan. But, when I gave it more data (such as image uploads) it would frequently give me the “Sorry, something went wrong” error message and I ended up switching back to ChatGPT. For the VSCode suggestions, because it didn’t know the actual math or variables I was trying to implement, these suggestions would be pretty random and incorrect so I didn’t use it for any large-scale coding tasks. I did find it quite helpful when there was repetition involved: for example when I wrote the implementation to find g (compliance) for the solid design area, it could generalize that to the void and uniform cases by taking my code and changing a few variable names.

As for ChatGPT, I was surprised at how well it handled syntax, even for Julia specific packages like *Gridap.jl* or *Gmsh*, again as long as a similar implementation was provided. So, I used ChatGPT to debug some issues that were related to the package language. I was stuck on an issue where the fixed end was displacing, so relative errors between adjoint and finite difference gradients were huge and there was negative compliance. When I gave ChatGPT just my code (the Jupyter notebook), it gave me a bunch of suggestions that did not do anything. However, when I provided it a tutorial code from *Gridap* and the accompanying mesh file, it was quickly able to identify the issue in the way I’d created my design file.

ChatGPT was also able to make my paper-writing much more efficient by converting things into LaTeX. So, I could give it a screenshot of an output printed in my Jupyter notebook and it could give me the styled table to copy straight into Overleaf—completely eliminating the time it would take me to

retype everything, get the correct table formatting etc. This is shown on the screenshot on the next page.

```
np = 35
===== Quick Comparison =====
g adjoint = 13.199886277656898
g FD      = 13.163864290873258
FD / adjoint g ratio = 0.9972710380357888
vol adjoint = 0.39998042973208353
vol FD      = 0.3999236518214883
FD / adjoint volume ratio = 0.999858048278429
```

what is a good way to include this data in a paper

Thought for a couple of seconds >

A good paper version should **not** include the terminal screenshot. Convert it into a compact validation table and explain what it demonstrates.

For example:

```
LaTeX
\begin{table}[h]
\centering
\caption{Comparison of objective and volume constraint values computed using the adjoint-based gradient and finite difference}
\label{tab:adjoint_fd_check}
\begin{tabular}{lcc}
\hline
Quantity & Adjoint & Finite difference \\
\hline
Compliance objective & $g$ & 13.1999 & 13.1639 \\
Volume fraction & 0.4000 & 0.3999 \\
\hline
Ratio: FD / adjoint objective & \multicolumn{2}{c}{0.9973} \\
Ratio: FD / adjoint volume & \multicolumn{2}{c}{0.9999}
\end{tabular}
\end{table}
```

ChatGPT prompt and output to generate a table from screenshotted data. Do note that there was context given before this specific prompt—such as the fact that I was using LaTeX and to not use an excessive amount of lines (like having every term of an equation on a separate line in code). Other than that, relatively simple prompts like this one were still quite effective.

REFERENCES

- [1] M. P. Bendsøe and N. Kikuchi, “Generating optimal topologies in structural design using a homogenization method,” *Computer Methods in Applied Mechanics and Engineering*, vol. 71, pp. 197–224, Nov. 1988.
- [2] C. Wang, Z. Zhao, M. Zhou, O. Sigmund, and X. S. Zhang, “A comprehensive review of educational articles on structural and multidisciplinary optimization,” *Structural and Multidisciplinary Optimization*, vol. 64, pp. 2827–2880, Nov. 2021.
- [3] O. Sigmund and K. Maute, “Topology optimization approaches: A comparative review,” *Structural and Multidisciplinary Optimization*, vol. 48, pp. 1031–1055, Dec. 2013.
- [4] D. Ha and J. Carstensen, “Automatic hyperparameter tuning of topology optimization algorithms using surrogate optimization,” *Structural and Multidisciplinary Optimization*, vol. 67, no. 9, p. 157, 2024.
- [5] J. Bezanon, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A Fresh Approach to Numerical Computing,” *SIAM Review*, vol. 59, pp. 65–98, Jan. 2017.
- [6] C. Geuzaine and J.-F. Remacle, “Gmsh: a three-dimensional finite element mesh generator with built-in pre-and post-processing facilities,” *International Journal for Numerical Methods in Engineering*, vol. 79, no. 11, pp. 1309–1331, 2009.
- [7] S. Badia and F. Verdugo, “Gridap: An extensible Finite Element toolbox in Julia,” *Journal of Open Source Software*, vol. 5, p. 2520, Aug. 2020.
- [8] M. Tarek, “Topopt.jl: An efficient and high-performance package for topology optimization of continuum structures in the julia programming language,” in *Proceedings of the 13th World Congress of Structural and Multidisciplinary Optimization*, 2019.
- [9] Z. J. Wegert, J. Manyer, C. N. Mallon, S. Badia, and V. J. Challis, “GridapTopOpt.jl: a scalable Julia toolbox for level set-based topology optimisation,” *Structural and Multidisciplinary Optimization*, vol. 68, p. 22, Jan. 2025.
- [10] M. P. Bendsøe, “Optimal shape design as a material distribution problem,” *Structural Optimization*, vol. 1, pp. 193–202, Dec. 1989.
- [11] M. J. Turner, R. W. Clough, H. C. Martin, and L. J. Topp, “Stiffness and deflection analysis of complex structures,” *Journal of the Aeronautical Sciences*, vol. 23, pp. 805–823, September 1956.
- [12] K. Svanberg, “The method of moving asymptotes—a new method for structural optimization,” *International Journal for Numerical Methods in Engineering*, vol. 24, pp. 359–373, Feb. 1987.
- [13] S. G. Johnson, “The NLOpt nonlinear-optimization package,” <https://github.com/stevengj/nlopt>, 2007.
- [14] T. T. Banh, Q. X. Lieu, J. Lee, J. Kang, and D. Lee, “A robust dynamic unified multi-material topology optimization method for functionally graded structures,” *Structural and Multidisciplinary Optimization*, vol. 66, p. 75, Apr. 2023.
- [15] M. Innes, “Don’t unroll adjoint: Differentiating ssa-form programs,” *CoRR*, vol. abs/1810.07951, 2018.
- [16] B. S. Lazarov and O. Sigmund, “Filters in topology optimization based on Helmholtz-type differential equations,” *International Journal for Numerical Methods in Engineering*, vol. 86, pp. 765–781, May 2011.
- [17] F. Wang, B. S. Lazarov, and O. Sigmund, “On projection methods, convergence and robust formulations in topology optimization,” *Structural and Multidisciplinary Optimization*, vol. 43, pp. 767–784, June 2011.
- [18] P. Dunning and F. Wein, “Automatic projection parameter increase for three-field density-based topology optimization,” *Structural and Multidisciplinary Optimization*, vol. 68, p. 33, Feb. 2025.
- [19] M. P. Bendsøe and O. Sigmund, “Material interpolation schemes in topology optimization,” *Archive of Applied Mechanics (Ingenieur Archiv)*, vol. 69, pp. 635–654, Nov. 1999.
- [20] J. C. Steuben, J. G. Michopoulos, A. P. Iliopoulos, and A. J. Birnbaum, “Functional Performance Tailoring of Additively Manufactured Components via Topology Optimization,” in *Volume 1: 37th Computers and Information in Engineering Conference*, (Cleveland, Ohio, USA), p. V001T02A026, American Society of Mechanical Engineers, Aug. 2017.

- [21] T. Zegard and G. H. Paulino, “Toward GPU accelerated topology optimization on unstructured meshes,” *Structural and Multidisciplinary Optimization*, vol. 48, pp. 473–485, Sept. 2013.
- [22] N. Aage, E. Andreassen, and B. S. Lazarov, “Topology optimization using PETSc: An easy-to-use, fully parallel, open source topology optimization framework,” *Structural and Multidisciplinary Optimization*, vol. 51, pp. 565–572, Mar. 2015.
- [23] T. Besard, C. Foket, and B. De Sutter, “Effective extensible programming: Unleashing Julia on GPUs,” *IEEE Transactions on Parallel and Distributed Systems*, 2018.